Uma introdução à Evolução Diferencial

[Nome do Autor][1]

[1]Programa de Pós-Graduação em Informática (PPGI)
Universidade Federal do Espírito Santo (UFES) – Vitória, ES – Brasil

[email]

Abstract. *Differential Evolution (DE) is a population-based metaheuristic for global optimization over continuous spaces, proposed by Storn and Price in 1995. New candidate solutions are built by adding the scaled difference between two population members to a third one, followed by binomial crossover and a greedy one-to-one selection. This paper reviews the origin and main variants of DE, describes the classic DE/rand/1/bin algorithm and how it balances exploration and exploitation, and illustrates it on the two-dimensional Rastrigin function, a multimodal benchmark with 121 local minima.*

Resumo. *A Evolução Diferencial (ED) é uma meta-heurística populacional para otimização global em espaços contínuos, proposta por Storn e Price em 1995. Novas soluções candidatas são geradas somando a um membro da população a diferença ponderada entre outros dois, seguida de um cruzamento binomial e de uma seleção gulosa um a um. Este artigo apresenta a origem e as principais variantes da ED, descreve o algoritmo clássico DE/rand/1/bin e o modo como ele equilibra diversificação e intensificação, e o ilustra na função de Rastrigin em duas dimensões, um problema multimodal com 121 mínimos locais.*

1. Introdução

Muitos problemas de engenharia e ciência podem ser formulados como a minimização de uma função objetivo $f : \mathbb{R}^D \to \mathbb{R}$ de variáveis contínuas. Quando f é não linear, não diferenciável ou multimodal, métodos de gradiente e buscas diretas locais, como a de Nelder e Mead, tendem a parar no primeiro mínimo local encontrado, pois só aceitam movimentos que melhoram a solução corrente [Storn and Price 1997]. Meta-heurísticas populacionais, como os algoritmos genéticos e as estratégias evolutivas, reduzem esse risco ao manter várias soluções ao mesmo tempo.

A Evolução Diferencial (ED, ou DE, do inglês *Differential Evolution*) foi proposta por Rainer Storn e Kenneth Price em um relatório técnico do International Computer Science Institute, em Berkeley [Storn and Price 1995]. O método surgiu das tentativas de Price de resolver um problema de ajuste de polinômios de Chebyshev proposto por Storn [Das and Suganthan 2011]. Em 1996, a ED participou da primeira competição internacional de otimização evolutiva (1st ICEO, durante a conferência IEEE ICEC'96) e foi o algoritmo evolutivo mais rápido entre os inscritos [Price and Storn 1996]. No ano seguinte, o artigo publicado no *Journal of Global Optimization* consolidou o método: comparada a variantes de recozimento simulado, a outros algoritmos evolutivos e a um método de

equações diferenciais estocásticas em três conjuntos de funções de teste, a ED exigiu menos avaliações da função objetivo na maioria dos casos [Storn and Price 1997]. Em 2005, os autores reuniram a teoria e a prática do método em um livro [Price et al. 2005].

O estado da arte concentra-se na adaptação automática dos parâmetros de controle e das estratégias de mutação. O jDE codifica F e CR em cada indivíduo e os deixa evoluir junto com a solução [Brest et al. 2006]; o SaDE escolhe entre várias estratégias de mutação conforme o sucesso recente de cada uma [Qin et al. 2009]; o JADE introduziu a mutação *current-to-pbest* com um arquivo externo de soluções descartadas [Zhang and Sanderson 2009]; o SHADE adapta F e CR a partir de uma memória histórica de sucessos [Tanabe and Fukunaga 2013]; e o L-SHADE, que acrescenta a redução linear do tamanho da população, venceu a competição de otimização mono-objetivo do IEEE CEC 2014 [Tanabe and Fukunaga 2014]. Revisões abrangentes dessas variantes são apresentadas em [Das and Suganthan 2011] e [Das et al. 2016].

Entre as aplicações da ED estão o projeto de filtros digitais [Storn 1996], eletromagnetismo, sistemas de potência, engenharia química, projeto mecânico e aprendizado de máquina [Das and Suganthan 2011, Das et al. 2016, Cornell University 2026]. A Seção 2 descreve o algoritmo, suas variantes e parâmetros; a Seção 3 aplica a ED à função de Rastrigin; e a Seção 4 apresenta as conclusões.

2. Definição e descrição da meta-heurística

A ED trabalha com uma população de NP vetores de parâmetros D-dimensionais, $\mathbf{x}_{i,G}$, $i = 1, \ldots, NP$, em que G indica a geração. O tamanho da população não muda durante a execução. A população inicial é sorteada com distribuição uniforme dentro dos limites de cada variável, $x_{j,i,0} = x_j^{\min} + \mathrm{rand}_j(0,1)\,(x_j^{\max} - x_j^{\min})$, de forma a cobrir todo o espaço de busca [Storn and Price 1997, Baeldung 2026]. A cada geração, cada vetor da população assume o papel de *vetor-alvo* e passa por três operações: mutação, cruzamento e seleção.

Mutação. Para cada vetor-alvo $\mathbf{x}_{i,G}$, gera-se um *vetor mutante* somando a um vetor-base a diferença ponderada entre dois outros membros da população:

$$\mathbf{v}_{i,G+1} = \mathbf{x}_{r_1,G} + F \cdot (\mathbf{x}_{r_2,G} - \mathbf{x}_{r_3,G})\,, \tag{1}$$

em que $r_1, r_2, r_3 \in \{1, \ldots, NP\}$ são índices sorteados, distintos entre si e diferentes de i (por isso $NP \geq 4$), e $F > 0$ é o *fator de escala*, que controla a amplitude da perturbação. Assim, a direção e o tamanho do passo vêm das diferenças entre os próprios membros da população, e não de uma distribuição fixa, como nas estratégias evolutivas.

Cruzamento. Para aumentar a diversidade, o mutante é combinado com o alvo, coordenada a coordenada, formando o *vetor experimental* $\mathbf{u}_{i,G+1}$:

$$u_{j,i,G+1} = \begin{cases} v_{j,i,G+1}, & \text{se } \mathrm{rand}_j \leq CR \text{ ou } j = j_{rand}, \\ x_{j,i,G}, & \text{caso contrário,} \end{cases} \qquad j = 1, \ldots, D, \tag{2}$$

em que $\mathrm{rand}_j \in [0,1]$ é sorteado para cada coordenada, $CR \in [0,1]$ é a *constante de cruzamento* e j_{rand} é um índice sorteado que garante que ao menos uma coordenada venha do mutante. O número de coordenadas herdadas do mutante segue aproximadamente uma distribuição binomial, daí o nome *cruzamento binomial*.

Algoritmo 1 Evolução Diferencial clássica (DE/rand/1/bin)

Entrada: f, limites $[\mathbf{x}^{\min}, \mathbf{x}^{\max}]$, $NP \geq 4$, F, CR, $G_{\max}$

1: Sortear $\mathbf{x}_{i,0}$ uniformemente nos limites e avaliar $f(\mathbf{x}_{i,0})$, $i = 1, \ldots, NP$
2: **para** $G = 0, 1, \ldots, G_{\max} - 1$ **faça**
3: **para** $i = 1, \ldots, NP$ **faça**
4: Sortear $r_1 \neq r_2 \neq r_3 \neq i$ em $\{1, \ldots, NP\}$ e j_{rand} em $\{1, \ldots, D\}$
5: $\mathbf{v} \leftarrow \mathbf{x}_{r_1,G} + F\,(\mathbf{x}_{r_2,G} - \mathbf{x}_{r_3,G})$ ▷ mutação
6: $u_j \leftarrow v_j$ se $\mathrm{rand}_j \leq CR$ ou $j = j_{rand}$; senão $u_j \leftarrow x_{j,i,G}$ ▷ cruzamento
7: $\mathbf{x}_{i,G+1} \leftarrow \mathbf{u}$ se $f(\mathbf{u}) < f(\mathbf{x}_{i,G})$; senão $\mathbf{x}_{i,G+1} \leftarrow \mathbf{x}_{i,G}$ ▷ seleção

Saída: o melhor vetor da população final

Seleção. O vetor experimental compete apenas com o seu próprio alvo, em uma seleção gulosa:

$$\mathbf{x}_{i,G+1} = \begin{cases} \mathbf{u}_{i,G+1}, & \text{se } f(\mathbf{u}_{i,G+1}) < f(\mathbf{x}_{i,G}), \\ \mathbf{x}_{i,G}, & \text{caso contrário.} \end{cases} \tag{3}$$

Assim, ocorrem NP competições por geração e o valor de f associado a cada posição da população nunca piora. O Algoritmo 1 resume o método. Como no pseudocódigo original [Storn and Price 1997], a nova população é montada separadamente e só substitui a atual ao final da geração.

Variantes. Storn e Price propuseram a notação DE/x/y/z [Storn and Price 1997], em que x indica o vetor-base (*rand*, um vetor aleatório, ou *best*, o melhor da população), y é o número de vetores diferença e z é o tipo de cruzamento (*bin*, binomial, ou *exp*, exponencial, que copia um bloco contíguo de coordenadas). O esquema das Equações 1–3 é o DE/rand/1/bin. Outras variantes comuns são o DE/best/1, com $\mathbf{v} = \mathbf{x}_{best} + F(\mathbf{x}_{r_1} - \mathbf{x}_{r_2})$; o DE/best/2, com $\mathbf{v} = \mathbf{x}_{best} + F(\mathbf{x}_{r_1} + \mathbf{x}_{r_2} - \mathbf{x}_{r_3} - \mathbf{x}_{r_4})$; e o DE/current-to-best/1, com $\mathbf{v} = \mathbf{x}_i + F(\mathbf{x}_{best} - \mathbf{x}_i) + F(\mathbf{x}_{r_1} - \mathbf{x}_{r_2})$. As variantes que usam o melhor vetor convergem mais rápido, mas correm mais risco de convergência prematura em funções multimodais [Das and Suganthan 2011].

Parâmetros de controle. Segundo [Storn and Price 1997], uma escolha razoável é NP entre $5D$ e $10D$; $F = 0{,}5$ é um bom valor inicial, e valores menores que $0{,}4$ ou maiores que 1 raramente funcionam bem; para CR, $0{,}1$ é uma primeira escolha segura, mas vale testar $0{,}9$ ou $1{,}0$ em busca de uma convergência rápida. Se a população convergir prematuramente, deve-se aumentar F e/ou NP. Valores baixos de CR funcionam bem em funções separáveis; valores altos, quando as variáveis interagem [Das and Suganthan 2011].

Diversificação e intensificação. O equilíbrio entre explorar o espaço de busca e refinar as boas soluções surge da própria estrutura do algoritmo. No início, a população está espalhada, os vetores diferença são longos e a mutação produz saltos grandes (*diversificação*); o vetor-base aleatório da variante *rand* reforça esse efeito, e a seleção um a um preserva diversidade, pois cada posição da população forma uma linhagem própria. À medida que a seleção gulosa descarta as soluções piores, a população se concentra nas regiões promissoras, as diferenças entre os vetores diminuem e o passo da mutação encolhe automaticamente (*intensificação*). Em regiões planas da função ocorre o inverso: as dis-

Tabela 1. Uma iteração da DE/rand/1/bin para um vetor-alvo da população inicial ($F = 0{,}5$, $CR = 0{,}9$).

Vetor	x_1	x_2	$f(\mathbf{x})$
$\mathbf{x}_i$ (alvo)	2,805	−0,626	31,887
$\mathbf{x}_{r_1}$ (base)	0,559	−4,467	59,363
$\mathbf{x}_{r_2}$	−2,796	1,739	28,664
$\mathbf{x}_{r_3}$	2,643	−1,490	45,419
$\mathbf{x}_{r_2} - \mathbf{x}_{r_3}$	−5,439	3,229	–
$\mathbf{v}$ (mutante)	−2,161	−2,852	–
rand_j	0,374	0,987	–
$\mathbf{u}$ (experimental)	−2,161	−0,626	26,770

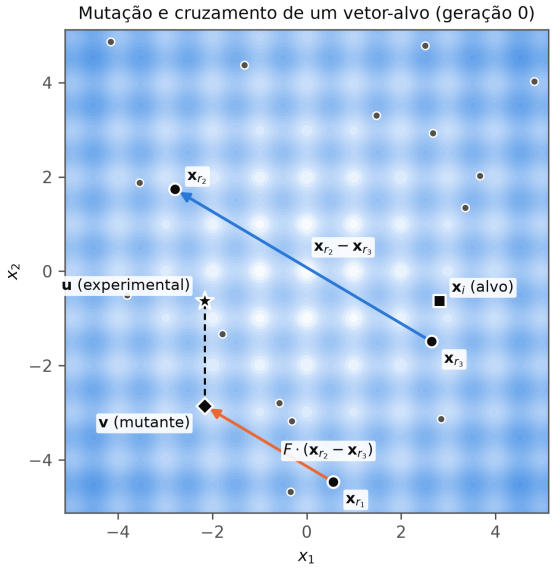


Figura 1. Vetores da Tabela 1: diferença (azul), passo $F(\mathbf{x}_{r_2} - \mathbf{x}_{r_3})$ (laranja) e cruzamento (tracejado).

tâncias entre os vetores aumentam, o que evita um avanço lento [Storn and Price 1997]. Os parâmetros F, CR e NP regulam, respectivamente, a amplitude dos passos, quantas coordenadas mudam e quanta diversidade é mantida.

Exemplo numérico. A Tabela 1 e a Figura 1 mostram uma iteração para um vetor-alvo da população inicial do experimento da Seção 3 ($F = 0{,}5$, $CR = 0{,}9$). No cruzamento, $\mathrm{rand}_1 = 0{,}374 \leq CR$, e a primeira coordenada veio do mutante; como $\mathrm{rand}_2 = 0{,}987 > CR$ e $j_{rand} = 1$, a segunda foi herdada do alvo. Como $f(\mathbf{u}) = 26{,}770 < f(\mathbf{x}_i) = 31{,}887$, o vetor experimental substitui o alvo na próxima geração.

3. Exemplo de aplicação da meta-heurística

3.1. O problema

Para ilustrar o funcionamento da ED, escolheu-se a minimização da função de Rastrigin,

$$f(\mathbf{x}) = 10D + \sum_{j=1}^{D} \left[x_j^2 - 10\cos(2\pi x_j)\right], \qquad \mathbf{x} \in [-5{,}12;\ 5{,}12]^D, \tag{4}$$

com $D = 2$. A função combina uma parábola com uma ondulação cossenoidal (Figura 2): há um mínimo local perto de cada ponto de coordenadas inteiras, 121 no domínio, e um único mínimo global, $f(0{,}0) = 0$, o que a torna um teste clássico de otimização global. Ela integra o conjunto de testes de [Storn and Price 1997], lá com domínio $[-600, 600]$; aqui usou-se o domínio usual para permitir visualizar a população.

3.2. Implementação

O experimento foi implementado em Python com a classe `DE` da biblioteca pymoo, versão 0.6.2 [Blank and Deb 2020], em um *notebook* Jupyter que também gera as figuras deste

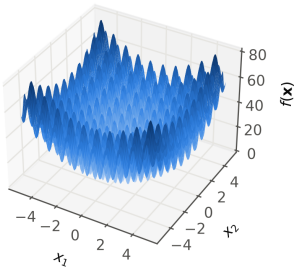


Figura 2. Função de Rastrigin ($D = 2$).

Tabela 2. Configuração do experimento.

Parâmetro	Valor	Argumento no `DE` do pymoo
Estratégia	DE/rand/1/bin	`variant="DE/rand/1/bin"`
População	$NP = 20$ $(10D)$	`pop_size=20`
Fator de escala	$F = 0{,}5$	`F=0.5, jitter=False`
Cruzamento	$CR = 0{,}9$	`CR=0.9`
Mutação extra	desligada	`prob_mut=0.0`
Gerações	$G_{\max} = 100$	`("n_gen", 101)`
Semente	42	`seed=42`

artigo e uma animação da população. A configuração (Tabela 2) segue as recomendações da Seção 2.

Alguns padrões do pymoo diferem da ED clássica e foram alterados: a variante padrão é a DE/best/1/bin, o CR padrão é $0{,}2$ e, após o cruzamento, a biblioteca aplica uma mutação polinomial extra (probabilidade $0{,}1$), que foi desligada por não fazer parte do algoritmo original. No pymoo, uma coordenada do mutante que sai do domínio é sorteada entre o vetor-base e o limite violado, e o papel de j_{rand} é cumprido sorteando uma coordenada do mutante só quando nenhuma passa no teste $\mathrm{rand}_j \leq CR$. Como a inicialização conta como primeira geração, 101 gerações do pymoo equivalem a 100 gerações da ED, ou $NP \times 101 = 2020$ avaliações de f.

3.3. Resultados

A Figura 3 mostra a população em quatro instantes da execução. Na geração 0, os 20 indivíduos estão espalhados pelo domínio e ocupam 19 bacias de atração diferentes (contadas pelo ponto de coordenadas inteiras mais próximo de cada indivíduo). Na geração 5, o melhor valor caiu de $25{,}0$ para $3{,}37$, mas a população ainda ocupa 14 bacias; na geração 15, ela se concentra perto da origem (12 bacias) e, na geração 35, restam 3. A partir da geração 42, todos os indivíduos estão na bacia do mínimo global e o restante da execução é de intensificação: na geração 100, a melhor solução foi $\mathbf{x}^* = (-2{,}49 \times 10^{-9};\ -2{,}40 \times 10^{-9})$, com $f(\mathbf{x}^*) = 3{,}55 \times 10^{-15}$. O limiar $f < 10^{-6}$ foi atingido na geração 61, após 1240 avaliações de f.

A Figura 4 resume a execução. O melhor valor de f cai em patamares, pois só muda quando algum vetor experimental supera o melhor indivíduo, enquanto a média da população cai de forma contínua. A dispersão da população, medida como a distância média dos indivíduos ao centroide, cai de $4{,}02$ para $1{,}8 \times 10^{-8}$. Como os vetores diferença são formados pelos próprios indivíduos, essa é também a escala dos passos da mutação, que a ED reduziu em mais de oito ordens de grandeza sem nenhum parâmetro adicional. A queda abrupta perto da geração 40 coincide com a eliminação dos últimos indivíduos que estavam em bacias vizinhas à do mínimo global.

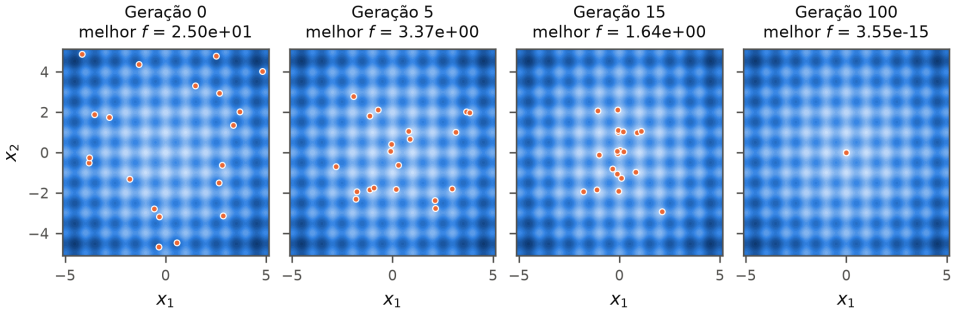


Figura 3. População da ED (pontos laranja) nas gerações 0, 5, 15 e 100 sobre as curvas de nível da função de Rastrigin.

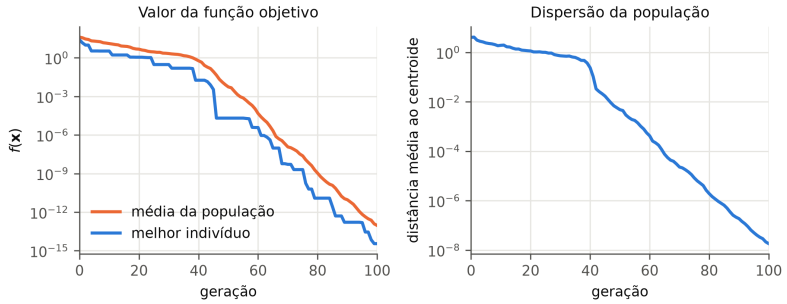


Figura 4. Convergência da ED na função de Rastrigin: valor de f do melhor indivíduo e média da população (à esquerda) e dispersão da população (à direita), em escala logarítmica.

4. Principais conclusões

A Evolução Diferencial é uma meta-heurística populacional simples: seu núcleo cabe em poucas linhas de código e depende de apenas três parâmetros de controle, para os quais existem boas regras práticas. Ela não exige derivadas, trata naturalmente problemas multimodais e é fácil de paralelizar. Sua característica mais marcante é a mutação baseada em vetores diferença, que ajusta o tamanho do passo à distribuição da população e produz a transição da diversificação para a intensificação observada na Seção 3, em que o método encontrou o mínimo global da função de Rastrigin, entre 121 mínimos locais, com 1240 avaliações até atingir $f < 10^{-6}$.

Entre as limitações, o desempenho depende de F, CR e da estratégia de mutação, cuja melhor escolha varia com o problema (o que motivou as variantes adaptativas), e a população pode estagnar ou convergir prematuramente, sobretudo se for pequena ou se a variante for gulosa, como a DE/best. A formulação original é voltada a variáveis contínuas sem restrições: problemas discretos ou com restrições exigem adaptações, como arredondamento ou funções de penalidade. O custo cresce com a dimensão, e não há garantia teórica de convergência para o ótimo global. Por fim, nenhuma meta-heurística é superior a todas as outras em todos os problemas [Wolpert and Macready 1997]; ainda assim, a simplicidade, a robustez e o bom desempenho em otimização contínua mantêm a ED entre as meta-heurísticas mais usadas e estudadas.

Referências

Baeldung (2026). Differential evolution algorithm. Baeldung on Computer Science. Disponível em: https://www.baeldung.com/cs/differential-evolution-algorithm. Acesso em: 27 set. 2026.

Blank, J. and Deb, K. (2020). pymoo: Multi-objective optimization in Python. *IEEE Access*, 8:89497–89509.

Brest, J., Greiner, S., Bošković, B., Mernik, M., and Žumer, V. (2006). Self-adapting control parameters in differential evolution: A comparative study on numerical benchmark problems. *IEEE Transactions on Evolutionary Computation*, 10(6):646–657.

Cornell University (2026). Differential evolution. Cornell University Computational Optimization Open Textbook. Disponível em: https://optimization.cbe.cornell.edu/index.php?title=Differential_evolution. Acesso em: 27 set. 2026.

Das, S., Mullick, S. S., and Suganthan, P. N. (2016). Recent advances in differential evolution – an updated survey. *Swarm and Evolutionary Computation*, 27:1–30.

Das, S. and Suganthan, P. N. (2011). Differential evolution: A survey of the state-of-the-art. *IEEE Transactions on Evolutionary Computation*, 15(1):4–31.

Price, K. and Storn, R. (1996). Minimizing the real functions of the ICEC'96 contest by differential evolution. In *Proceedings of the IEEE International Conference on Evolutionary Computation (ICEC'96)*, pages 842–844.

Price, K. V., Storn, R. M., and Lampinen, J. A. (2005). *Differential Evolution: A Practical Approach to Global Optimization*. Natural Computing Series. Springer, Berlin.

Qin, A. K., Huang, V. L., and Suganthan, P. N. (2009). Differential evolution algorithm with strategy adaptation for global numerical optimization. *IEEE Transactions on Evolutionary Computation*, 13(2):398–417.

Storn, R. (1996). Differential evolution design of an IIR-filter. In *Proceedings of the IEEE International Conference on Evolutionary Computation (ICEC'96)*, pages 268–273.

Storn, R. and Price, K. (1995). Differential evolution – a simple and efficient adaptive scheme for global optimization over continuous spaces. Technical Report TR-95-012, International Computer Science Institute, Berkeley, CA.

Storn, R. and Price, K. (1997). Differential evolution – a simple and efficient heuristic for global optimization over continuous spaces. *Journal of Global Optimization*, 11(4):341–359.

Tanabe, R. and Fukunaga, A. (2013). Success-history based parameter adaptation for differential evolution. In *Proceedings of the IEEE Congress on Evolutionary Computation (CEC)*, pages 71–78.

Tanabe, R. and Fukunaga, A. S. (2014). Improving the search performance of SHADE using linear population size reduction. In *Proceedings of the IEEE Congress on Evolutionary Computation (CEC)*, pages 1658–1665.

Wolpert, D. H. and Macready, W. G. (1997). No free lunch theorems for optimization. *IEEE Transactions on Evolutionary Computation*, 1(1):67–82.

Zhang, J. and Sanderson, A. C. (2009). JADE: Adaptive differential evolution with optional external archive. *IEEE Transactions on Evolutionary Computation*, 13(5):945–958.